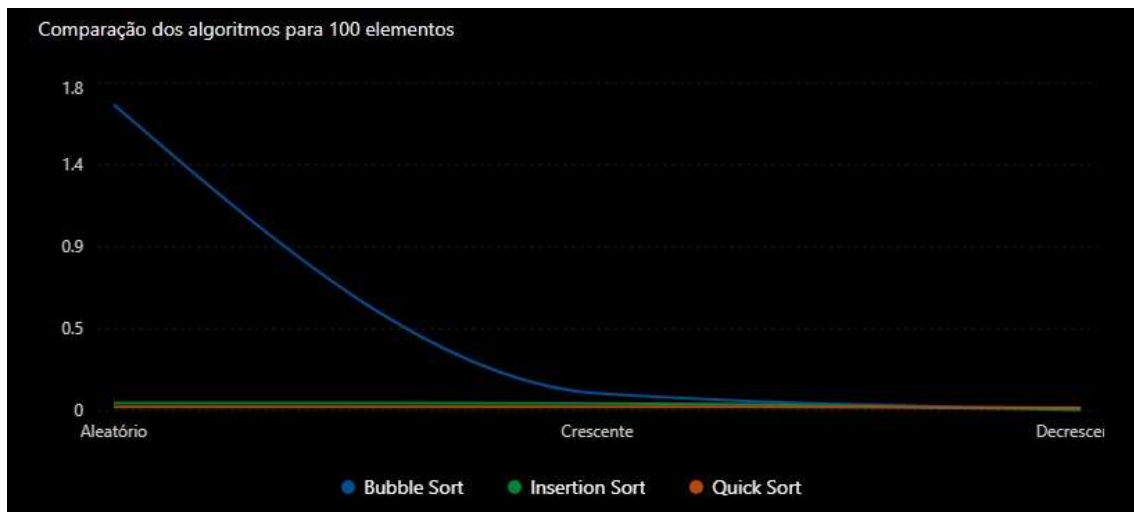


Relatório de Análise de Algoritmos de Ordenação

1. Conjunto de Dados: 100 Elementos

Tabela 1 - Conjuntos com 100 Elementos

Tipo de Dados	Bubble Sort (ms)	Insertion Sort (ms)	Quick Sort (ms)
Aleatório	1,6823	0,0421	0,0212
Crescente	0,0949	0,0405	0,0213
Decrescente	0,0089	0,0036	0,0146



Por se tratar de um volume muito pequeno de dados, todos os algoritmos executaram quase instantaneamente (abaixo de 2 ms). Nos cenários Aleatório e Crescente, o Quick Sort levou a melhor.

No entanto, no cenário Decrescente, nota-se uma inversão onde o Insertion e o Bubble Sort foram anormalmente mais rápidos que o Quick Sort. Esse fenômeno não ocorre por eficiência lógica, mas sim pelo Warmup da JVM (Java Virtual Machine): como o cenário decrescente rodou por último no programa, o compilador JIT do Java já havia otimizado a execução dos métodos na memória.

2. Conjunto de Dados: 1.000 Elementos

Tabela 2 - Conjuntos com 1000 Elementos

Tipo de Dados	Bubble Sort (ms)	Insertion Sort (ms)	Quick Sort (ms)
Aleatório	10,7306	1,6933	0,2525
Crescente	2,7380	0,0495	0,8704
Decrescente	0,5433	0,2124	0,5531

Tabela 2 - Conjuntos com 1000 Elementos

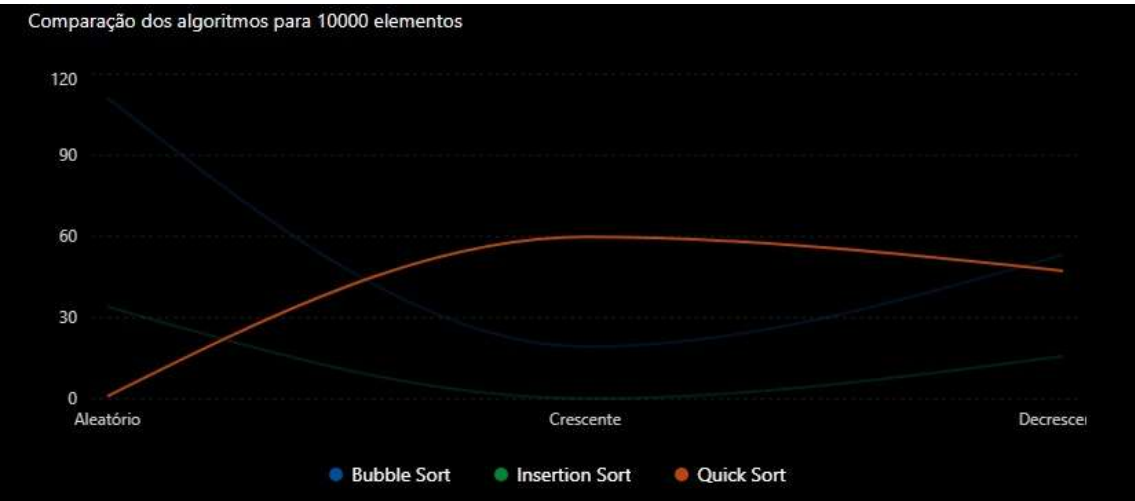
Tipo de Dados	Bubble Sort (ms)	Insertion Sort (ms)	Quick Sort (ms)
Aleatório	10,7306	1,6933	0,2525
Crescente	2,7380	0,0495	0,8704
Decrescente	0,5433	0,2124	0,5531

Com o aumento do volume de dados para 1.000 elementos, as diferenças de complexidade começam a aparecer. No cenário Aleatório, o Quick Sort confirmou sua soberania, sendo 42 vezes mais rápido que o Bubble Sort.

A grande surpresa surge no cenário Crescente: o Insertion Sort superou o Quick Sort (0,0495 ms contra 0,8704 ms). Isso ocorre porque um vetor já ordenado representa o "melhor caso" teórico do Insertion Sort (complexidade $O(n)$), enquanto para o Quick Sort com pivô fixo no final, o cenário ordenado começa a engrenar o seu "pior caso" devido a partições desbalanceadas.

3. Conjunto de Dados: 10.000 Elementos

Tabela 3 - Conjuntos com 10000 Elementos			
Tipo de Dados	Bubble Sort (ms)	Insertion Sort (ms)	Quick Sort (ms)
Aleatório	111,3049	34,0396	0,9741
Crescente	19,2368	0,0739	59,8554
Decrescente	53,2899	15,7387	47,3471



Este cenário reflete perfeitamente a teoria assintótica das estruturas de dados. No caso Aleatório, o Quick Sort humilhou os concorrentes, terminando em menos de 1 ms, enquanto o Bubble Sort ultrapassou os 111 ms (reflexo de sua complexidade quadrática $O(n^2)$).

Contudo, nos cenários Crescente e Decrescente, o "ponto cego" do Quick Sort ficou escancarado. Como o algoritmo foi obrigado a usar o último elemento como pivô, ele gerou subvetores totalmente desproporcionais ao pegar dados já ordenados, degradando sua complexidade para $O(n^2)$. Por isso, no cenário crescente, o Quick Sort foi o pior de todos (59,85 ms), perdendo feio para o Insertion Sort (0,0739 ms), que apenas varreu o vetor confirmando a ordenação.